# Cahier des charges

Détection d'intrusions réseau par machine learning
Optimisation d'hyperparamètres sur NSL-KDD

## 1. Contexte et état actuel

**Contexte :** sécurité des réseaux informatiques. Les réseaux sont exposés à des attaques (déni de service, scans de ports, accès non autorisés). Un système de détection d'intrusions (IDS) surveille le trafic et signale les connexions suspectes.

### État actuel et irritants

- Les règles écrites à la main ne détectent pas les nouvelles attaques.
- Le trafic est volumineux et les fausses alertes fatiguent les analystes.
- Les modèles de machine learning dépendent fortement du réglage de leurs hyperparamètres, souvent fait au hasard.

**Pourquoi ce projet :** trouver, de façon méthodique, la meilleure configuration d'un modèle de détection en comparant plusieurs méthodes d'optimisation.

## 2. Objectifs et indicateurs (KPI)

**Objectif stratégique :** obtenir un modèle qui distingue trafic normal et attaque, dont la configuration optimale est justifiée par la comparaison d'au moins 3 méthodes d'optimisation, d'ici la date de remise du projet.

| Objectif mesurable | Indicateur | Critère de succès |
|---|---|---|
| Détecter les attaques | Recall (classe attaque) | Amélioré par rapport à la baseline |
| Limiter les fausses alertes | Precision | Pas de baisse excessive du Recall |
| Équilibre global | F1-score (métrique principale) | Modèle optimisé > baseline sur le test |
| Comparer les méthodes | Score et temps d'optimisation | 3 méthodes comparées dans un tableau |
| Reproductibilité | Résultats identiques à chaque exécution | random_state=42 fixé |

## 3. Périmètre (In / Out)

| Inclus (In) | Hors périmètre (Out) |
|---|---|
| Dataset NSL-KDD (train et test) | Capture de trafic réseau réel |
| Classification binaire normal / attaque | Déploiement en production |
| 4 algorithmes avec paramètres par défaut | Interface web ou application |
| Optimisation par 3 méthodes | Détection en temps réel |
| Comparaison, choix du modèle, évaluation finale | Deep learning (réseaux de neurones) |
| Rapport écrit | Classification multi-classe (extension optionnelle) |

## 4. Parcours utilisateurs et exigences fonctionnelles

### Personas

- **Analyste sécurité :** veut être alerté des vraies attaques.
- **Enseignant (MOA) :** veut vérifier la rigueur de la méthode.
- **Étudiant (MOE) :** veut un projet clair et reproductible.

### User stories

| ID | En tant que... | Je veux... | Afin de... | Critère d'acceptation |
|---|---|---|---|---|
| US1 | Analyste | que chaque connexion soit classée normale ou attaque | réagir aux intrusions | Une prédiction par connexion |
| US2 | Analyste | un fort Recall sur les attaques | ne pas rater d'intrusion | Recall affiché et comparé à la baseline |
| US3 | Enseignant | voir tous les essais dans un seul tableau | comparer les méthodes | Tableau avec score et temps par expérience |
| US4 | Enseignant | que le test set reste intact jusqu'à la fin | éviter la fuite de données | Test utilisé une seule fois, à l'étape finale |
| US5 | Étudiant | pouvoir relancer tout le projet | reproduire les résultats | README et environment.yml fournis |

## 5. Spécifications et contraintes techniques

- **Langage et outils :** Python, scikit-learn, XGBoost, Optuna, Jupyter (Weka en option).
- **Validation :** validation croisée à 5 plis sur le jeu d'entraînement.
- **Performance :** chaque optimisation doit tourner sur un ordinateur portable standard en un temps raisonnable (à mesurer et reporter).
- **Qualité des données :** aucune fuite de données, normalisation ajustée sur le train uniquement.
- **Sécurité et données :** dataset public et anonymisé, pas de données personnelles (RGPD non concerné).
- **Versionnement :** Git, hébergé sur GitHub, données exclues du dépôt.

Les détails (algorithmes, méthodes, métriques) sont en annexe A.

## 6. Gouvernance, rôles et ressources

| Rôle | Personne |
|---|---|
| MOA / sponsor | Enseignant |
| MOE | Étudiant(e) |

### Matrice RACI

| Activité | Étudiant | Enseignant |
|---|---|---|
| Choix du contexte et du dataset | R | A |
| Préparation des données | R/A | I |
| Expériences et optimisation | R/A | C |
| Choix du modèle final | R | A |

| Activité | Étudiant | Enseignant |
|---|---|---|
| Rédaction du rapport | R/A | C |
| Validation finale | I | A |

*R = réalise, A = décide, C = consulté, I = informé.* **Ressources :** ordinateur personnel, environnement conda, dataset NSL-KDD.

## 7. Planning, méthodologie et assurance qualité

**Méthode :** itérative (Agile simplifié), une étape à la fois, avec un commit Git à la fin de chaque étape.

| Jalon | Étape | Livrable | Date |
|---|---|---|---|
| J1 | Environnement, repo Git, chargement des données | Notebook 01_exploration | __/__/____ |
| J2 | Préparation des données | Notebook 02_preparation | __/__/____ |
| J3 | Baseline (paramètres par défaut) | Tableau des scores | __/__/____ |
| J4 | Optimisation (Grid, Random, Optuna) | Meilleurs hyperparamètres | __/__/____ |
| J5 | Comparaison de toutes les expériences | Tableau et graphiques (results/) | __/__/____ |
| J6 | Évaluation finale sur le test set | Métriques finales | __/__/____ |
| J7 | Rapport | Fichier dans reports/ | __/__/____ |

### Assurance qualité et tests

- Vérifier les dimensions des données après chaque transformation.
- Fixer random_state=42 partout.
- Vérifier l'absence de fuite (le test set n'est jamais utilisé avant J6).
- Relire le code et le rapport avant la remise.

## 8. Budget, risques et clauses

**Budget :** 0 € (outils open source). Seul coût : le temps de calcul des optimisations.

### Risques

| Risque | Probabilité | Plan d'action |
|---|---|---|
| Optimisation trop longue | Moyenne | Réduire la grille, limiter les essais Optuna, utiliser n_jobs=-1 |
| Surapprentissage (test différent du train) | Moyenne | Validation croisée, comparer scores CV et test |
| Classes déséquilibrées | Faible | Suivre F1 et Recall, pas seulement l'accuracy |
| Fuite de données | Moyenne | Pipeline scikit-learn, test set isolé |
| Perte de travail | Faible | Commits Git réguliers, push sur GitHub |

### Clauses

- **Propriété intellectuelle :** le dataset NSL-KDD est public, à citer dans le rapport.
- **Réversibilité :** tout le code est versionné et l'environnement exporté (environment.yml).

# Annexe A : Détails techniques

**Algorithmes (baseline) :** Régression logistique, Arbre de décision, Random Forest, XGBoost.

### Méthodes d'optimisation

| Méthode | Principe |
|---|---|
| Grid Search | Teste toutes les combinaisons d'une grille |
| Random Search | Teste des combinaisons tirées au hasard |
| Optuna (bayésien) | Choisit les essais suivants selon les résultats précédents |

**Métriques :** Accuracy, Precision, Recall, F1-score, ROC-AUC, matrice de confusion, temps d'exécution.

# Annexe B : Structure du projet

```
intrusion-check/
|-- data/         données (non versionnées)
|-- notebooks/    exploration, baseline, optimisation
|-- src/          code réutilisable
|-- results/      tableaux et graphiques
|-- reports/      rapport final
|-- environment.yml
`-- README.md
```

# Annexe C : Installation

```
conda create -n intrusion-detection python=3.11 -y
conda activate intrusion-detection
conda install -c conda-forge pandas numpy scikit-learn matplotlib seaborn jupyter -y
pip install optuna xgboost
```

# Annexe D : Résultats

*À compléter : meilleur modèle, hyperparamètres optimaux, scores finaux.*